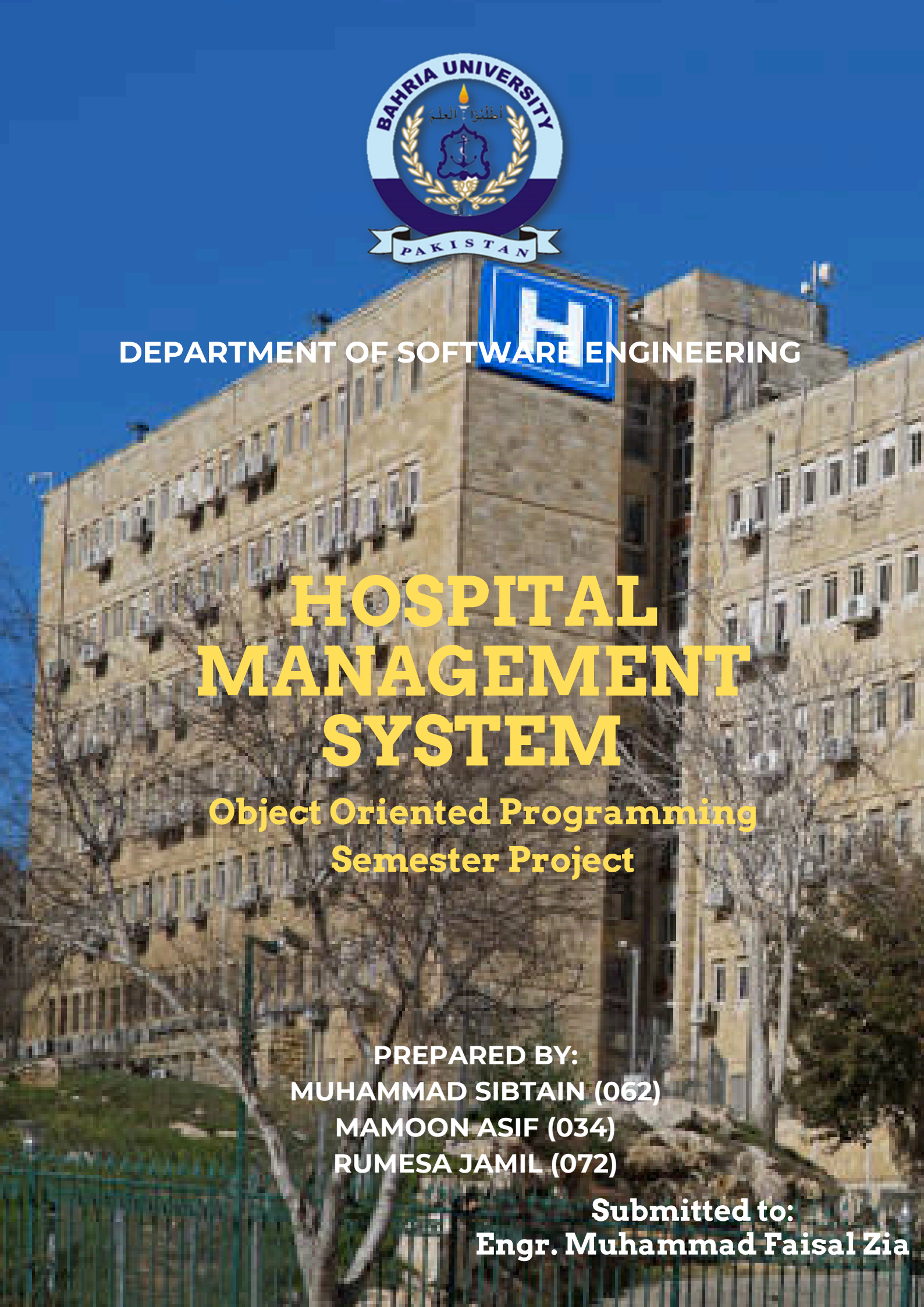


DEPARTMENT OF SOFTWARE ENGINEERING

HOSPITAL MANAGEMENT SYSTEM

Object Oriented Programming
Semester Project

PREPARED BY:
MUHAMMAD SIBTAIN (062)
MAMOON ASIF (034)
RUMESA JAMIL (072)

Submitted to:
Engr. Muhammad Faisal Zia

TABLE OF CONTENTS

Summary

The Hospital Management System (HMS) is a desktop graphical application developed in pure C++ utilizing the native Windows API (<windows.h>). While traditional console-based configurations present barrier entries for non-technical users, this implementation provides an accessible layout featuring dual-column layouts, responsive menu arrays, and dialog alert fields.

Architecturally, the application uses advanced Object-Oriented Design principles. By moving away from basic structural matrices to specialized template abstractions like std::vector, memory allocations expand naturally as records accumulate. System stability is achieved via strong string token validations and low-level boundary checks. When application loops shut down, the persistent file-handling subsystem locks down the states, parsing active structural data safely down into plain text sheets (patients.txt, doctors.txt, and appointments.txt) to protect information across system restarts.

Github:

https://github.com/MSibtain062/hospital-management-system.git

Introduction & Problem Statement

2.1 Context and Rationalization

- Medical institutions generate massive data streams hourly, ranging from incoming patient biometric arrays to doctor scheduling details. Relying on paper registries or static spreadsheet cells reduces administrative workflow efficiency, delays data retrieval, and leaves sensitive medical information exposed to physical wear or accidental deletion.
- This project solves these issues by shifting clinical operations to a specialized local environment. Using native low-level C++ compilation frameworks ensures fast data processing speeds, low memory usage, and structural reliability.

2.2 Operational Vulnerabilities of Legacy Frameworks

- Manual or semi-automated data filing structures exhibit key structural flaws:
- Input Vulnerabilities: Hand-written records allow unreadable entries, incorrect patient IDs, or missing clinical notes.
- Retrieval Latency: Sifting sequentially through offline documents results in long processing queues.
- Scheduling Collisions: Basic tracking sheets lack overlapping-time protection, causing multiple patient appointments to conflict on the same doctor's index.

2.3 System Scope & Core Software Specifications

The system automates and organizes administrative duties into three operational pillars:

1. Patient Registry Operations: Creating non-overlapping record tokens, indexing names, validating ages, and allowing administrative deletion routines.
2. Medical Staff Directory Management: Registering practitioners alongside distinct specialization tags and tracking clinical experience fields.
3. Relational Scheduling Interface: A unified validation engine that pairs active, checked patient keys against registered doctor IDs on specified calendar matrices.

Application GUI Architecture

3.1 Win32 Native API Implementation Context

Rather than relying on heavy cross-platform layout engines (like Qt) or web dependencies, this architecture interfaces directly with the operating system kernel via the native Microsoft Win32 Software Development Kit (SDK). This choice maintains a small executable file size, ensures fast processing, and bypasses external driver requirements.

3.2 Dynamic Text Entry Fields & Custom Modal Windows (GUI Input Text)

Data entry shifts away from the unsafe default console inputs (cin) into localized, modal dialogue boxes. When an administrator triggers an operation like "Add Patient", the program invokes a custom dialog component wrapper (GuiInputText). This routine registers a temporary child layout frame (G_INPUT_CLASS), updates the layout focus directly inside a single-line input control, and captures text fields via a private pointer structure (InputState), returning control safely back to the parent frame upon confirmation.

3.4 Windows Message Loops & Asynchronous Event Processing

The application lifecycle runs asynchronously using an event-driven operating system paradigm. The execution entry point (main) maintains a live query loop (GetMessage) that captures peripheral events. These system notifications are forwarded directly via DispatchMessage to a specialized callback controller function (MainWndProc). Inside this loop, identifier numbers (2001 through 2012) handle specific user actions, translating button clicks into direct operations on the underlying Hospital class database without blocking the user interface.

Object-Oriented Design & System Methodology

4.1 Global Encapsulation & Pure Vector Allocations

To avoid data corruption across open files, all operational databases live inside the Hospital controller class as private, encapsulated dynamic arrays (vector<Patient>, vector<Doctor>, vector<Appointment>). Individual entity states cannot be directly accessed or changed from outside the class boundary. Instead, interactions must go through secure getter functions, protecting data consistency during active editing sessions.

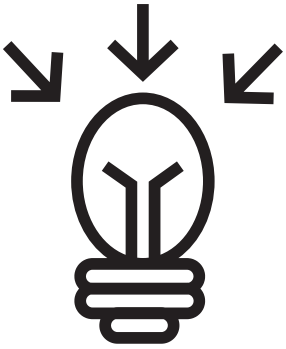
4.2 Class Hierarchy Trees

The system structures its data using a clear base-to-derived public inheritance relationship:

- Base Class (Person): An abstract class that stores foundational attributes common to all individuals in the system, including id, name, age, and gender.
- Derived Class (Patient): Inherits from Person and extends the structure by adding patient-specific clinical attributes: disease and bloodGroup.
- Derived Class (Doctor): Inherits from Person and extends the structure by adding professional staff attributes: specialization and experience.

4.3 Virtual Interfaces & Polymorphic Framework Bindings

The base class enforces structure by defining a pure virtual function (virtual void displayInfo() = 0), which makes Person an abstract type that cannot be instantiated on its own. The derived child objects provide their own specific display overrides. In the GUI code, formatting methods extract these variables polymorphically via string stream collectors (ostringstream), keeping data processing logic cleanly separated from interface rendering code.



Conclusion

The Hospital Management System successfully achieves all goals outlined in the laboratory design specification. By building the application with native Win32 SDK functions rather than standard command-line routines, the system provides an accessible interface for users without sacrificing performance. The design prevents memory leaks, secures data boundaries through inheritance fields, and guarantees data security via reliable file serialization.